

PPL Week-10

Rachit Takate – 230962294 – B54

I. Write a program in CUDA to perform matrix multiplication using 2D Grid and 2D Block.

```
#include <cuda.h>
#include <stdio.h>

#define M 3
#define N 4
#define L 3

__global__ void matMul(float *matA, float *matB, float *matRes, uint m, uint n,
uint l) {
    uint j = blockDim.x * blockIdx.x + threadIdx.x,
    i = blockDim.y * blockIdx.y + threadIdx.y;

    if(i < m && j < l) {
        float acc = 0;
        for(uint k = 0; k < n; k++)
            acc += matA[k + n * i] * matB[j + l * k];
        matRes[j + l * i] = acc;
    }
}

void printMatf(float *mat, uint m, uint n) {
    for(uint i = 0; i < m; i++) {
        for(uint j = 0; j < n; j++)
            printf("%.0f ", mat[i * n + j]);
        printf("\n");
    }
}

int main() {
    float matA[M][N] = {
        {0, 1, 2, 3},
        {4, 5, 6, 7},
        {8, 9, 10, 11}
    },
    matB[N][L] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9},
        {10, 11, 12}
    },
    matRes[M][L];
    float *dA, *dB, *dRes;

    size_t sA = sizeof(float),
    sB = sA * N * L,
    sRes = sA * M * L;
    sA *= M * N;

    cudaMalloc((void **) &dA, sA);
    cudaMalloc((void **) &dB, sB);
    cudaMalloc((void **) &dRes, sRes);

    cudaMemcpy(dA, matA, sA, cudaMemcpyHostToDevice);
    cudaMemcpy(dB, matB, sB, cudaMemcpyHostToDevice);
```

```

    matMul<<<dim3((M + 15) / 16, (N + 15) / 16), dim3(16, 16)>>>(dA, dB, dRes,
M, N, L);
    cudaMemcpy(matRes, dRes, sRes, cudaMemcpyDeviceToHost);
    printf("Result:\n");
    printMatf((float *) matRes, M, L);

    cudaFree(dA);
    cudaFree(dB);
    cudaFree(dRes);
    printf("Rachit 54\n");
    return 0;
}

```

Output

```

(base) mca@computinglab26-26:~/230962294/PP Lab/Week10$ nvcc Q1.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week10$ ./a.out
Result:
48 54 60
136 158 180
224 262 300

```

II. Write a program in CUDA to improve the performance of 1D parallel convolution using constant memory.

```

#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>

__constant__ float dKer[1024];

__global__ void conv1D(float *arr, float *res, uint mLen, uint len) {
    uint i = blockDim.x * blockIdx.x + threadIdx.x;

    if(i < len) {
        int start = i - mLen / 2;
        float val = 0;

        for (int j = 0; j < mLen; j++) {
            int idx = start + j;
            if (idx >= 0 && idx < len) {
                val += arr[idx] * dKer[j];
            }
        }

        res[i] = val;
    }
}

void printArrf(float *arr, uint len) {
    for (uint i = 0; i < len; i++)
        printf("%.0f ", arr[i]);
    printf("\n");
}

int main() {
    float *arr, *ker, *res;
    float *dArr, *dRes;
    int n, m;

    printf("Array size: ");
    scanf("%d", &n);
    printf("Mask size: ");
    scanf("%d", &m);
}

```

```

size_t nSize = sizeof(float) * n;
size_t mSize = sizeof(float) * m;

arr = (float *) malloc(nSize);
ker = (float *) malloc(mSize);
res = (float *) malloc(nSize);

for (int i = 0; i < n; i++) {
    arr[i] = rand() % 10;
    if (i < m)
        ker[i] = rand() % 10;
}

printf("Array:\n");
printArrf(arr, n);
printf("Kernel:\n");
printArrf(ker, m);

cudaMalloc((void **) &dArr, nSize);
cudaMalloc((void **) &dRes, nSize);

cudaMemcpy(dArr, arr, nSize, cudaMemcpyHostToDevice);
cudaMemcpyToSymbol(dKer, ker, mSize);

conv1D<<<(n + 255) / 256,>>>(dArr, dRes, m, n);
cudaMemcpy(res, dRes, nSize, cudaMemcpyDeviceToHost);

printf("Result:\n");
printArrf(res, n);

cudaFree(dArr);
cudaFree(dRes);
free(arr);
free(ker);
free(res);

printf("Rachit 54\n");
return 0;
}

```

Output

```

(base) mca@computinglab26-26:~/230962294/PP Lab/Week10$ nvcc Q2.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week10$ ./a.out
Array size: 128
Mask size: 3
Array:
3 7 3 6 2 9 1 2 7 0 9 3 6 0 6 2 6 1 8 7 9 2 0 2 3 7 5 9 2 2 8 9 7 3 6 1 2 9 3 1 9 4 7 8 4 5 0 3 6 1 0 6 3 2 0 6 1 5 5 4 7 6 5 6 9 3 7 4 5 2 5 4 7 4 4 3 0 7 8 6 8 8 4 3 1 4 9 2 0 6 8 0
2 6 6 4 9 5 0 4 8 7 1 7 2 7 2 2 6 1 0 6 1 5 9 4 9 0 9 1 7 7 1 1 5 9 7 7
Kernel:
6 5 5
Result:
50 68 87 58 91 62 69 51 47 87 60 99 48 66 40 76 47 81 81 128 97 64 22 25 62 78 112 85 74 62 97 128 104 87 53 51 61 72 74 68 71 109 99 102 93 49 45 45 53 41 36 45 61 28 42 35 66 56 75
85 89 97 91 105 96 104 73 87 59 65 57 85 79 82 59 39 53 75 112 118 116 108 83 44 43 71 79 64 42 70 121 103 94 72 86 101 94 79 50 60 99 88 82 51 87 57 62 52 47 41 36 35 66 76 95 119 69
99 50 94 76 82 52 36 76 110 124 77
Rachit 54

```

III. Write a CUDA program to perform tiled 1D convolution operation on the input array N of size width using the mask array, M of size mask_width, to produce the resultant array P of size width.

```

#include <stdio.h>
#include <stdlib.h>
#include <cuda.h>

```

```

__global__ void conv1D(float *arr, float *ker, float *res, uint mLen, uint len)
{
    uint i = blockDim.x * blockIdx.x + threadIdx.x;

    if(i < len) {
        __shared__ float tile[35];

```

```

    int n = mLen / 2;

    int haloL = i - blockDim.x;
    if(threadIdx.x >= blockDim.x - n)
        tile[threadIdx.x - blockDim.x + n] = (haloL < 0) ? 0 : arr[haloL];

    tile[n + threadIdx.x] = arr[blockIdx.x * blockDim.x + threadIdx.x];

    int haloR = i + blockDim.x;
    if(threadIdx.x < n)
        tile[threadIdx.x + blockDim.x + n] = (haloR >= len) ? 0 :
arr[haloR];

    __syncthreads();

    float val = 0;
    for (uint j = 0; j < mLen; j++)
        val += tile[threadIdx.x + j] * ker[j];

    res[i] = val;
}

void printArrf(float *arr, uint len) {
    for (uint i = 0; i < len; i++)
        printf("%.0f ", arr[i]);
    printf("\n");
}

int main() {
    float *arr, *ker, *res;
    float *dArr, *dKer, *dRes;
    int n, m;

    printf("Array size: ");
    scanf("%d", &n);
    printf("Mask size: ");
    scanf("%d", &m);

    size_t nSize = sizeof(float) * n;
    size_t mSize = sizeof(float) * m;

    arr = (float *) malloc(nSize);
    ker = (float *) malloc(mSize);
    res = (float *) malloc(nSize);

    for (int i = 0; i < n; i++) {
        arr[i] = rand() % 10;
        if (i < m)
            ker[i] = rand() % 10;
    }

    printf("Array:\n");
    printArrf(arr, n);
    printf("Kernel:\n");
    printArrf(ker, m);

    cudaMalloc((void **) &dArr, nSize);
    cudaMalloc((void **) &dKer, mSize);
    cudaMalloc((void **) &dRes, nSize);

    cudaMemcpy(dArr, arr, nSize, cudaMemcpyHostToDevice);
    cudaMemcpy(dKer, ker, mSize, cudaMemcpyHostToDevice);

```

```

conv1D<<<(n + 31) / 32, 32>>>(dArr, dKer, dRes, m, n);
cudaMemcpy(res, dRes, nSize, cudaMemcpyDeviceToHost);

printf("Result:\n");
printArrf(res, n);

cudaFree(dArr);
cudaFree(dKer);
cudaFree(dRes);
free(arr);
free(ker);
free(res);

printf("Rachit 54\n");
return 0;
}

```

Output

```

(base) mca@computinglab26-26:~/230962294/PP Lab/Week10$ nvcc Q3.cu
(base) mca@computinglab26-26:~/230962294/PP Lab/Week10$ ./a.out
Array size: 100
Mask size: 5
Array:
3 7 3 6 9 2 7 0 9 3 6 0 6 2 6 1 8 7 9 2 0 2 3 7 5 9 2 2 8 9 7 3 6 1 2 9 3 1 9 4 7 8 4 5 0 3 6 1 0 6 3 2 0 6 1 5 5 4 7 6 5 6 9 3 7 4 5 2 5 4 7 4 4 3 0 7 8 6 8 8 4 3 1 4 9 2 0 6 8 9 2 6
6 4 9 5 0 4 8 7
Kernel:
6 5 5 2 1
Result:
32 62 89 107 104 105 108 68 99 72 105 62 76 53 86 70 104 101 132 99 71 35 44 81 98 118 97 99 87 114 140 117 91 66 72 68 83 96 83 93 129 112 112 96 61 58 47 59 56 44 49 67 41 49 50 80
71 95 102 105 118 112 118 114 117 85 96 73 80 75 97 90 88 66 61 75 95 136 138 127 115 89 61 63 75 85 84 67 90 131 121 110 89 109 111 98 16 43 74 99
Rachit 54

```